Event Delivery Pipeline Replatforming

Project description: Lines 242, 244 and 246

FIELD OF SCIENCE OR TECHNOLOGY	CLAIM PERIOD	PREPARED BY	LENGTH (242 / 244 / 246)
Software engineering (distributed systems)	FY2025	Judah Paemka	299 / 585 / 308 words

About this sample. *This narrative is based on an anonymized, composite project. Client names, internal system names and some figures have been changed or generalized to protect client confidentiality. It demonstrates technical writing and is not a filed claim.*

LINE 242

Scientific or Technological Uncertainties

What scientific or technological uncertainties did you attempt to overcome?

299 of 350 words

Our real-time customer data platform processes approximately 2 million events per second and more than 2 trillion events per month. A Kafka-based broadcast layer and an in-house orchestrator deliver events to warehouses and activation endpoints. During FY2025, we sought to replace the broker, introduce company-level cohorts and move ingestion orchestration to Temporal. These changes had to preserve per-partition ordering, keep p99 end-to-end delivery latency below 5 seconds at peak load, and prevent a delayed batch from stalling other pipeline stages.

We had operating knowledge of the incumbent topology and access to Kafka and Redpanda guidance on partitions, replication and consumer rebalancing. We also drew on DynamoDB sharding patterns, idempotent delivery methods and Temporal's workflow, activity and retry model. Those methods addressed individual design problems. What we could not establish from them was how the revised delivery paths, shared synchronization workload and asynchronous stage handoffs would behave together under our load and recovery conditions. The uncertainty concerned preserving the platform's operating constraints while these dependencies changed.

For the broker, we needed to determine whether a 1,024-partition, three-replica cluster with 72-hour retention could sustain peak throughput and preserve ordering without amplifying backpressure across consumers. Batch-ingested events no longer passed through the legacy Kafka cluster, so we also needed a delivery path that could serve broadcast consumers during normal operation and backfill. Company-level membership introduced a different access pattern into synchronization built around users. We did not know whether upstream membership resolution against a separate DynamoDB model could coexist with the user path without contention, latency regression or incorrect membership. Finally, removing the orchestrator's wait for downstream acknowledgment changed when change data capture (CDC) batches could advance. We needed to determine whether the append-only streams would remain intact and whether Temporal retries could handle stage handoffs without duplicate or overlapping batch submissions.

LINE 244

Work Performed

What work did you perform in the tax year to overcome the scientific or technological uncertainties described in Line 242?

585 of 700 words

We sequenced the FY2025 experiments so that one variable changed at a time. Ordering, latency and fault isolation remained the acceptance criteria as we examined the broker, cohort and orchestration changes.

We began with the broker. We hypothesized that a Redpanda-compatible cluster with 1,024 partitions, three replicas and 72-hour retention could absorb peak load with ordering intact, and deployed it in a dedicated Kubernetes environment. We mirrored production traffic for three weeks, replayed 1.5 times observed peak load and forced leadership changes through controlled broker restarts. Throughput held, per-partition ordering was preserved, and consumer lag recovered within 90 seconds of each restart. Consumer-path analysis nevertheless showed that batch-ingested events no longer passed through the legacy cluster and could not reach broadcast consumers through that route. Broker capacity alone therefore did not establish delivery continuity. We then hypothesized that direct file-based reads from the event store could supply those events. Delivery and backfill were restored, but sustained backfill raised p99 latency to 11 seconds, above the 5-second target.

For company-level cohorts, we hypothesized that resolving membership upstream during behavioural marker identification would allow a separate company entity to coexist with user-level synchronization. We built a company-keyed DynamoDB table, integrated it into the Celery workers and ran concurrent load tests. Writes keyed only by company identifier created hot partitions and throttling for accounts above roughly 40,000 users. Spreading large companies across 16 shard suffixes and aggregating on read eliminated the observed throttling. Membership was correct for every user in the test companies, with no measurable regression in the user-level path.

We next hypothesized that the CDC append-only streams would remain intact without the scheduler's blocking wait for downstream batch acknowledgment. We removed the wait and monitored the streams for breakage. Full-stream stalls caused by a single delayed batch no longer occurred, and approximately 90 percent of traffic moved through the decoupled path. This supported independent scheduling of subsequent batches under the tested conditions. The change increased batch-ingestion API calls, so we grouped storage keys under one batch identifier, polled at batch level and decomposed failed batches during fallback. That reduced redundant calls without regression.

For orchestration, we hypothesized that Temporal workflows and activities could represent our stage handoffs, and that its default retry semantics would preserve correctness when a stage failed mid-handoff. The proof of concept preserved data handoff semantics and sustained production-like job volumes in cross-team load tests. Failure injection disproved the second part. Heartbeat-based retries resubmitted in-flight batches, and because the downstream batch service assigns a new deduplication key to every submission, it treated each retry as new work. Overlapping batches appeared in about 0.4 percent of retried jobs. We separated heartbeat handling from status checks, made submission idempotent on a stable batch key and revised retry policies. Repeating the failure tests produced no duplicates, showing that retry safety depended on how the downstream service assigned deduplication identity, not on the orchestrator alone.

Supporting load tests also examined deduplication and export scheduling. We hypothesized that the deduplication ceiling came from thread and port architecture rather than node capacity; a redesigned architecture achieved equivalent throughput with 16 times fewer threads. Classifying export jobs by cadence

and scheduling by priority and age allowed high-cadence jobs to meet their service level objectives without additional infrastructure. These tests identified constraints that added capacity alone would not have addressed efficiently.

The steady-state tests supported the revised broker, cohort and orchestration designs. Sustained backfill still exceeded the latency target, so the read path required further work. That investigation continues into FY2026.

LINE 246

Scientific or Technological Advancements

What scientific or technological advancements did you achieve or attempt to achieve as a result of the work described in Line 244?

308 of 350 words

The broker tests established that the proposed partition and replica configuration could sustain the tested throughput while preserving per-partition ordering, with consumer lag recovering within 90 seconds of controlled restarts. They also showed the limit of evaluating the broker on capacity alone. Batch-ingested events no longer traversed the legacy cluster, so broadcast consumers needed another route. File-based reads restored delivery, but their 11-second p99 latency during sustained backfill did not meet the 5-second target. We therefore established steady-state feasibility while leaving backfill performance unresolved.

Concurrent cohort tests identified the contention introduced by company-level writes. A company-only key concentrated writes for large accounts. Sharding those keys and aggregating on read supported correct membership across the test companies without measurable regression in user synchronization. This defined an access pattern under which the two identity models could coexist.

The CDC experiments showed that synchronous downstream acknowledgment was not required to preserve the tested append-only streams. In the previous configuration, a delayed batch could stop the whole stream. Independent scheduling removed those stalls, while batch-level submission and polling controlled the additional API work.

Failure injection distinguished retrying an activity from safely repeating its downstream submission. Default heartbeat-based retries are unsafe when the downstream service assigns a new deduplication key to every submission, because each retry is treated as new work and batches overlap. Stable batch keys, revised retry policies and separate heartbeat and status handling produced no duplicates in the repeated tests. This established the conditions under which stage handoffs can be retried safely on Temporal.

The supporting tests also located specific resource constraints in threading and scheduling, allowing equivalent throughput and timely exports without added infrastructure. Taken together, the experiments extended our knowledge of how the revised paths behaved under steady-state load and injected failures. They did not resolve delivery latency during sustained backfill, which remains the focus of FY2026 work.